SRL: A Formal Specification for Byte-Anchored Assertions

SRL-SPEC 1.0.0 — Formal specification.

Shri Narayan Justin Ram. Affiliation: Mushku Nobleworks. ORCID: 0009-0008-3593-2816

Copyright 2026 Mushku Nobleworks. Specification text: Creative Commons Attribution 4.0 International.

Sections 1–7 form the normative contract; §§8–10 are informative. The contract has no external normative dependency.

1. Scope, terminology, and notation

This contract specifies five operations over retained byte streams and stand-off reflection records. A **reflection record** is an assertion record containing an ordered nonempty sequence of byte-span references and an opaque statement. “Reflection” denotes neither a mathematical inverse nor a truth judgment. **Schema-blind** means that source-domain parsing, schema negotiation, normalization, and statement interpretation are not required. The library validates its own request and record structure.

A **library instance** fixes profile P and has logical state $\Sigma = (S,R)$. The **byte store** S maps addresses to retained bytes; the **catalogue** R maps identities to admitted bodies. **Stand-off** separation concerns roles and references: it neither requires unequal byte values nor separate physical stores. **Grounding** establishes the specified correspondence to retained bytes; **opaque** means uninterpreted by this contract, not secret.

MUST and **MUST NOT** express requirements on profiles and realizations claiming conformance. **MAY** permits behavior consistent with all requirements. Defined mathematical laws and transition rules are requirements even without those keywords.

$\mathrm{Seq}^+(X)$ denotes finite nonempty ordered sequences over X; $\mathrm{Map}(X,Y)$ denotes partial maps with finite domains. $\mathrm{dom}(M)$ is a map's domain. $M[a := v]$ adds or updates only that binding. $M \sqsubseteq N$ means $\mathrm{dom}(M) \subseteq \mathrm{dom}(N)$ and $N[a] = M[a]$ for every old key a. Byte equality includes length and every octet in order. Tuple and sequence equality includes every component, sequence length, order, and repetitions. Tagged alternatives are distinct without prescribing a wire representation.

$\mathrm{length}(b)$ is the number of bytes in b; ε is the empty byte sequence. For $0 \le i \le j \le \mathrm{length}(b)$, $b[i:j]$ is the zero-based half-open slice containing offsets i through $j-1$. A guarded map or slice expression is used only on a branch whose domain and bounds premises hold; a failed premise makes that guarded branch false. An unneeded expression in another disjunct is not evaluated. This convention makes the definitions well-defined and imposes no physical check order.

2. Domains and fixed profile

2.1 Domains and results

```
Byte    = {0,...,255}
Bytes   = Byte*
Nat     = {0,1,...}
Address = the identifier domain declared by P
Span    = Address × Nat × Nat × Address
Body    = Seq+(Span) × Bytes
State   = Map(Address,Bytes) × Map(Address,Body)

NotInCatalogue ∉ Body
LookupResult = Body ∪ {NotInCatalogue}

F = {InvalidRequest, Unavailable, InvalidSpan,
     Collision, IntegrityFailure, ResourceLimit}

Outcome(X) = {Success(x) | x ∈ X} ∪ {Failure(e) | e ∈ F}
```

Offsets and lengths are exact natural numbers. Empty sources and empty statements are valid byte values. F is exhaustive for this interface. `NotInCatalogue` is a distinct successful lookup

value, never a failure code.

2.2 Addressing and encoding laws

```
H : Bytes → Address
E : Body  → Bytes

∀ r,r' ∈ Body: E(r) = E(r') ⇔ r = r'

∃ D : image(E) → Body:
    ∀ r ∈ Body: D(E(r)) = r

identity(r) = H(E(r))
```

H and E MUST be deterministic under the fixed profile and depend only on their stated input and that profile. E retains the entire ordered target sequence, repetitions, and statement. D expresses exact recovery on `image(E)` and adds no public decoder operation. `E(r)` MAY be empty; injectivity permits this for at most one body.

H need not be injective or cryptographically secure. Equal identifiers do not establish equal bytes, including across instances. Identities are interpreted within their profile context; different profiles may agree or disagree on an identifier. No particular algorithm, wire format, or physical representation is prescribed. Changing a defining function or policy changes the profile; existing bindings cannot silently be reinterpreted under the changed profile.

2.3 Profile declaration and reading map

Every profile MUST supply behaviorally sufficient declarations for the following components. Labels alone are insufficient; declarations MUST satisfy the referenced laws.

Component	Required declaration	Governing clauses
Identifier domain and addressing	`Address`, deterministic total H, and the fixed context needed to interpret identifiers.	§§2.1–2.2, 3
Record encoding	Deterministic injective E and justification of exact recovery of every body component.	§§2.2, 3, 5.3
Equality and verification	Methods implementing exact logical equality; evidence and trust/fault assumptions establishing retained bytes, bindings, and catalogue answers.	§§1, 4.4, 7
Conditions	`Condition_P`, the complete relevant read, access, integrity, and resource facts for an operation, and their justified concrete interpretation.	§§4.1–4.2, 7.1
Assessment and failure selection	Rules sufficient to determine the observable outcome, including how relevant evidence and established applicable failures determine it.	§§4.2–4.3
Resources	Applicable limits, accounting, and refusal conditions for complete operations, including admission checks, encoding, and publication.	§§4.3, 5.3
Operating scope	Supported states, caller/access conditions, environmental and fault behavior, retention assumptions, and any supported concurrency or recovery behavior.	§§3, 7

Resource limits may restrict enabled operations without changing `Nat`, `Bytes`, or `Body`. A representable natural offset exceeding a declared implementation limit is not thereby a malformed abstract offset; the applicable resource failure is determined by §4. Arithmetic MUST NOT wrap, truncate, or silently change an abstract value.

3. Admissible logical state

For a body `r = (T,s)`, `targets(r) = T` and `s` is opaque. For `t = (a,i,j,h)`, all predicates below are relative to fixed P:

```
SpanOK(S,t) ⇔
    a ∈ dom(S)
    ∧ 0 ≤ i < j ≤ length(S[a])
    ∧ H(S[a][i:j]) = h

Compatible(S,b) ⇔
    H(b) ∉ dom(S) ∨ S[H(b)] = b

WF(S,R) ⇔
    (∀ a ∈ dom(S): H(S[a]) = a)
    ∧ (∀ c ∈ dom(R):
         c ∈ dom(S)
         ∧ S[c] = E(R[c])
         ∧ ∀ t ∈ targets(R[c]): SpanOK(S,t))

Admissible_P = {Σ ∈ State | WF(Σ)}
Σ₀ = (∅,∅)
WF(Σ₀)
```

These predicates concern retained logical values. Unavailability does not delete them. The contract defines operations from admissible states; it supplies no repair transition for an inadmissible state. A fresh instance starts at Σ_0; a supported retained start must preserve its admissible bindings under §7.

A span identifies an occurrence in a retained byte value, not an original file, observer, or acquisition event. Repeated text, repeated targets, overlap, and adjacency are permitted. The slice commitment h need not itself be a stored address. Matching it does not prove an unstated caller-intended quotation when unequal bytes collide under H. Content identity establishes neither origin, authorship, observation time, nor truth; referencing a record inherits none of its purported authority.

4. Requests, assessment, and failures

4.1 Request boundary and conditions

`Request` includes all candidate requests presented to the five operations, including malformed requests. Its structurally valid subset is exactly:

```
ValidRequest_P =
    {Put(b)          | b ∈ Bytes}
  ∪ {Select(a,i,j,h) | (a,i,j,h) ∈ Span}
  ∪ {Admit(T,s)      | (T,s) ∈ Body}
  ∪ {Fetch(a)        | a ∈ Address}
  ∪ {Lookup(a)       | a ∈ Address}
```

These are abstract forms, not a serialization. `Select(t)` and `Admit(r)` group their existing arguments; state and conditions are evaluation context, not additional caller arguments. Expressions requiring valid arguments MUST NOT treat malformed arguments as valid.

`Condition_P` is the declared domain of complete relevant read, access, integrity, and resource conditions. One `k ∈ Condition_P` applies to the complete operation, including every admission check. It may contain different facts for different reads; it does not assert that physical reads occur at one time or receive identical verdicts.

4.2 Total abstract assessment

For every admissible Σ, complete candidate request q, and declared k, `Step_P(Σ,q,k)` MUST determine exactly one result and post-state. Each operation is one atomic transition over that pre-state and request.

The profile MUST specify behaviorally sufficient assessment and failure-selection rules to determine the result for the same complete inputs. Error precedence alone is insufficient if differing established failures could change that result. Internal observation, check order, and short-circuiting MAY vary when they preserve the required result and effects; exhaustive discovery of all possible defects is not required. Assessment MUST establish either all prerequisites for the applicable success row in §5 or a nonempty set of applicable failures in §4.3. Only failures relevant to the operation may be established.

If the success prerequisites hold and no specified failure prevents success, success is required. Otherwise exactly one established applicable failure is selected by the profile. The returned failure does not assert that no other defect exists. Policy and conditions MUST NOT invent failures, waive required checks, or add source-domain parsing or statement interpretation as structural admission criteria. Independently declared access restrictions remain applicable.

4.3 Closed failure meanings

Failure	Applicable meaning
`InvalidRequest`	The candidate request is outside `ValidRequest_P`.
`Unavailable`	Required bytes or information cannot be obtained, including a requested source absent from S, denied access, or unavailable information needed for occupancy, exact comparison, or catalogue membership/nonmembership.
`InvalidSpan`	A structurally valid span has established invalid bounds, or verified retained source information establishes a failed slice commitment.
`Collision`	Proposed bytes differ from the bytes already retained at their proposed occupied address.
`IntegrityFailure`	Required integrity verification fails, including established divergence from retained bytes despite a matching digest, or inconsistent binding, body, or catalogue evidence.
`ResourceLimit`	The complete operation cannot meet the declared resource policy or limits.

No universal error precedence is imposed. A contradiction such as $i \geq j$ can be established without reading a source; trustworthy length information can establish a length-bound contradiction. A slice-commitment check needs sufficient verified retained source information and the selected bytes. Unknown information is not an established contradiction. Malformed requests cannot succeed; `InvalidRequest` is applicable, with selection governed by the declared assessment policy.

`Unavailable` can include established absence of required source bytes; it does not assert absence. A known vacant write address satisfies the vacancy part of compatibility and is not, on that account, an availability failure.

4.4 Evidence and exact results

Verification MUST justify the facts used by the operation under the declared trust and fault assumptions. A matching digest alone MUST NOT identify unequal byte values as equal. Every successful result MUST equal the value prescribed in §5; returned bytes and bodies MUST equal the corresponding retained logical values.

This is an observable obligation, not a claim that hashing detects every physical corruption. Failure to obtain required evidence is `Unavailable`; detected contrary integrity evidence is `IntegrityFailure`; a declared resource obstruction is `ResourceLimit`, subject to §4.2. Sufficient verified evidence is required; no universal physical read, comparison algorithm, recursive traversal, or whole-store scan is prescribed.

5. Successful transitions and effects

5.1 Operation table

Every row additionally requires §4's verification, access, and complete-operation resource prerequisites. Otherwise §4 determines the failure result.

Request	Logical guard and successful result	Post-state
`Put(b)`	Let `a = H(b)`. Require `Compatible(S,b)`; return `Success(a)`.	`(S[a := b], R)`
`Select(t)`	Require `SpanOK(S,t)`, sufficient verified retained-source information to justify it, and obtainable selected bytes verified equal to `S[a][i:j]`; return `Success(S[a][i:j])`.	Σ
`Admit(r)`	Let `b = E(r)`, `c = H(b)`. Every target meets the `Select` success conditions within this admission over the same pre-state and complete k; require `Compatible(S,b)`; return `Success(c)`.	`(S[c := b], R[c := r])`
`Fetch(a)`	Require `a ∈ dom(S)` and obtainable, verified retained bytes; return `Success(S[a])`.	Σ
`Lookup(a)`	With sufficient verified catalogue information, return `Success(R[a])` if `a ∈ dom(R)`, otherwise `Success(NotInCatalogue)`.	Σ

For `Select`, write `t = (a,i,j,h)`. All unmentioned bindings remain unchanged.

5.2 Reads and catalogue answers

Selection reads retained bytes; it does not reconstruct an unavailable source from its identifier. Exact source binding and interval correspondence remain required even when a realization uses a justified partial read or cached evidence.

`NotInCatalogue` establishes nonmembership only in this catalogue. Unavailable, inaccessible, incomplete, or unverified information MUST NOT become a successful negative. `Lookup` need not fetch each referenced source, fetch `E(r)` separately, or revalidate historical admission when sufficient trustworthy catalogue information establishes the exact result. Current access conditions still apply.

5.3 Admission and occupied addresses

Target checks in `Admit` are validation conditions, not mandatory separate public calls. Its resource policy covers the complete admission, including all checks, encoding, hashing, and publication; no budget is implicitly reset per target. Verification work may be shared while the retained target sequence keeps every position and repetition. One failed target rejects the entire body. Proposed encoding bytes do not supply an otherwise missing target in the pre-state.

Occupied-address compatibility requires exact equality. If required occupancy or comparison information cannot be verified, the write MUST fail under §4 rather than presume vacancy or equality. Equal existing bytes leave S unchanged. `Put(E(r))` alone never adds `r` to R. Under `WF`, exact S compatibility and injective E already prevent replacement of a different body; a separate physical `R[c]` read is not universally required.

5.4 Frame, failure, and repetition

```
Failure(e)                          ⇒ Σ' = Σ
operation ∈ {Select,Fetch,Lookup}   ⇒ Σ' = Σ
```

These obligations concern the logical maps, not elapsed work or private bookkeeping. Whenever repetition succeeds, identical `Put(b)` or `Admit(r)` returns the same identity and changes no existing entry. A later invocation may fail under changed read, access, integrity, or resource conditions. Re-admission does not bypass its guards.

6. Invariants and epistemic requirements

For every admissible pre-state and specified operation:

```
Step_P(Σ,q,k) = (Σ',v) ⇒ WF(Σ') ∧ S ⊑ S' ∧ R ⊑ R'
same Σ, q, P, k        ⇒ same Σ', v
```

Reachability from an admissible state is a finite sequence of these transitions. All such histories preserve earlier bindings. No acyclic-reference requirement is imposed.

ID	Requirement
R01 — Source neutrality	Within declared limits and subject to specified failures, `Put` MUST accept arbitrary finite bytes without source-domain parsing, schema negotiation, normalization, or rewriting. Request and record structure MAY be validated.
R02 — Exact retention	Stored bytes and admitted bodies MUST remain unchanged under these operations. Successful resolutions MUST match the fixed profile and retained values; digest equality MUST NOT replace exact equality, and conflicting occupied-address writes MUST be rejected.
R03 — Complete grounding	An admitted body MUST retain its exact ordered targets, repetitions, and opaque statement. Every target MUST satisfy admission conditions; one failed target MUST reject the whole body.
R04 — Epistemic separation	Source bytes, statements, and admission verdicts MUST retain distinct roles. Grounding MUST NOT imply semantic support, truth, authentication, permission, or authority to execute instructions contained in data.
R05 — Nonreplacement	Different bodies concerning the same source MAY coexist when their admission conditions permit. Agreement, disagreement, or interpretation MUST NOT authorize overwriting. Repeated successful admission of an identical body MUST be idempotent under fixed P.
R06 — Determinate outcomes	Results MUST be deterministic for the same complete state, request, profile, and conditions. Failure MUST preserve the pre-state. Missing information MUST NOT become established nonexistence; resource limits MUST NOT silently truncate a successful result.
R07 — Relationship fidelity	Relationships MUST preserve target positions and repeated occurrences. An absent relationship MUST NOT imply a negative semantic assertion.
R08 — Demonstrated conformance	A conformance claim MUST identify the complete profile and operating scope and justify this contract, including rejection and §7's realization obligations. A hash, stored record, or successful example alone is insufficient.

7. Realization and conformance

7.1 Sequential realization

Observations are returned values/failures and effects on retained bytes and admitted bodies. A realization MUST define one fixed abstraction α from its supported concrete states to admissible logical states. Every completed operation within the declared scope MUST be covered by the corresponding realization obligation. Fresh-empty initialization corresponds to Σ_0; supported continuation from retained state preserves its admissible bindings.

At sequential operation boundaries:

```
WF(α(z))

α(z) = Σ ∧ implementation(z,q,k) = (z',v)
    ⇒ Step_P(Σ,q,k) = (α(z'),v)
```

The abstraction, operating scope, and mapping from concrete conditions to k MUST be justified independently of the result being claimed. They MUST NOT redefine retained values or membership to excuse an incorrect return, corruption, or unavailability. Internal layout, caching, compression, and bookkeeping are unrestricted subject to these obligations. No public logging interface or abstraction of every intermediate machine instruction is required.

7.2 Concurrency and environmental transitions

If concurrency is supported, each finite invocation/response history MUST admit completion of zero or more pending invocations such that, after omitting the remaining pending invocations, the completed operations have an interpretation by the same atomic sequential transitions with the prescribed results and abstract effects. Real-time precedence is preserved: an operation that returned before another was invoked precedes it. Omitted invocations contribute no effects to that interpretation. Each admission uses one pre-state.

An overlapping call's invocation/response boundary states need not differ by only that call's transition. The sequential equation in §7.1 is not imposed across such overlapping boundaries.

Crash recovery or other environment-driven transitions claimed within scope require a corresponding preservation argument under the declared fault and retention assumptions. `WF` alone does not prove preservation of committed history. The preservation argument MUST account for every abstract recovery effect in the claimed history, including any permitted completion of pending operations; stored encodings alone do not establish prior admission. This contract supplies no recovery algorithm, deletion transition, or profile-migration procedure. An external change cannot silently be represented as a preserving continuation of the same instance when it loses or reinterprets retained bindings.

7.3 Evidence and enabled behavior

A conformance demonstration MUST address the applicable obligations below; its form and verification method are not prescribed.

Obligation	Evidence to justify
Profile laws and exact values	The declared domains, `H`, `E`, recovery, comparisons, and verified returned values satisfy §§1–5.
Initialization and retained starts	Supported starts map to admissible states and preserve the claimed retained history.
Success, refusal, and framing	Completed operations match all applicable transition results, including selected errors, atomic admission, and unchanged state on failure.
Deterministic assessment	The complete conditions and fixed assessment rules justify one outcome without choosing inputs retrospectively to fit it; permitted internal variation preserves that outcome.
Scope-dependent behavior	Claimed concurrency, environmental/fault behavior, and recovery satisfy §7.2 under stated assumptions.
Conditional realizability	Enabled successful transitions can be realized under the declared assumptions; absent handlers or refusal-only behavior are insufficient where success is enabled.

The sequential equation is a safety obligation over completed operations. Conditional realizability adds no scheduling fairness, execution-time bound, minimum capacity, or eventual-availability guarantee. Totality of the abstract step is not a proof of concrete termination. Finite examples alone do not establish general conformance; synthetic collision cases do not establish a production profile's security or realizability of that collision.

8. Worked conformance cases — informative

Cases instantiate the contract, not an additional wire format, digest corpus, test method, or precedence rule. Each case is independent unless its own steps say otherwise. Successful cases assume compatible addresses, sufficient verified information, and enabled access/resources. A named failure assumes it is the profile-selected established failure; every failure preserves both maps.

Byte literals are hexadecimal octets: `"abc" = 61 62 63`, `"aba" = 61 62 61`, `"a" = 61`, `"b" = 62`, `"c" = 63`; ε is empty.

Case	Preconditions and request	Prescribed result or consequence
T01 — Empty bytes	`Put(ε)`, then `Fetch(H(ε))`.	Both succeed with the address and ε, respectively. No nonempty span exists in the empty source.
T02 — Exact occurrence	Retain `"abc"` at `a`; request `Select(a,1,2,H("b"))`.	Exactly `"b"`, without mutation. Admitting that span with any byte statement establishes grounding, not truth.
T03 — Order and repeats	Retain `"aba"` at `a`; `t0=(a,0,1,H("a"))`, `t2=(a,2,3,H("a"))`; admit `([t2,t0,t2],ε)`.	The body retains that exact three-element sequence and empty statement.
T04 — Multiple sources	Retain `"abc"` at `a` and `"aba"` at `d`; admit targets `(a,1,2,H("b"))` and `(d,2,3,H("a"))`.	Both targets are justified in the same pre-state; their order remains significant. Address compatibility is an explicit premise.
T05 — Storage versus admission	`Put(E(r))` succeeds before `r` is admitted; all targets are later available for `Admit(r)`.	Initial `Lookup(identity(r))` returns `NotInCatalogue`; successful admission adds R membership without changing the compatible encoded bytes.
T06 — Repetition and nested bytes	Repeat an enabled successful `Put(b)` or `Admit(r)`. For an admitted `r` with nonempty `E(r)`, select `(c,0,length(E(r)),c)`, where `c=identity(r)`.	Repetition preserves bindings and identity; the whole-encoding selection returns `E(r)`. Empty encodings have no nonempty interval.
T07 — InvalidRequest	Submit `Admit([],ε)` or an abstract negative offset.	`InvalidRequest` is applicable because the request is outside the declared domains. No state change.
T08 — Unavailable	Fetch an address established absent from S, or request a lookup for which no sufficient permitted catalogue evidence can be obtained.	`Unavailable` is applicable. The failed lookup cannot return `NotInCatalogue`. A backend outage alone is insufficient if trustworthy alternative evidence remains available.
T09 — InvalidSpan: bounds	In `"abc"`, admit a valid `[1,2)` target followed by a `[2,4)` target.	`InvalidSpan` is applicable; the entire admission fails without partial publication. An interval `i ≥ j` is independently invalid even when the source is unavailable.
T10 — InvalidSpan: commitment	Obtain verified source `"abc"`; choose `h' ∈ Address` with `h' ≠ H("b")`; select `(a,1,2,h')`.	`InvalidSpan` is applicable. This conditional case requires such an `h'`; a profile need not supply every illustrative witness.
T11 — Collision	Retain `x`; propose unequal `y` with `H(x)=H(y)`, through `Put(y)` or a valid `Admit(r)` with `E(r)=y`.	`Collision` is applicable; the incumbent and all catalogue bindings remain unchanged. A separate synthetic profile may supply the witness.
T12 — IntegrityFailure	Required obtained bytes or catalogue evidence are established inconsistent with the retained logical value, including a known same-digest byte substitution.	`IntegrityFailure` is applicable. A digest match cannot authorize a different successful result. The case assumes the mismatch is established under the declared verifier.
T13 — ResourceLimit	Each target could be checked alone, but the complete admission exceeds its declared resource budget.	`ResourceLimit` is applicable; no partial storage, catalogue insertion, or truncated success.
T14 — Changed conditions	Admit `r`, then deny a required read or exhaust an applicable resource before replay.	The replay may fail while the earlier bindings and `identity(r)` remain unchanged.
T15 — Multiple failures	Invalid bounds and unavailable required information are both established.	The profile's assessment/selection policy chooses the error. No universal choice follows from this document.
T16 — Pending write	From Σ_0, `Put(b)` commits but its response is pending; a concurrent `Fetch(H(b))` returns `b`. These are the only invocations.	The history interpretation includes an appropriate completion of the pending write. Omitting the write omits its effect.

T17 — Profile context	Two profiles use identical H/E but different lawful error policies; another profile changes an identity function.	The first pair may yield identical identities; the latter may differ. Different profile names neither prove equality nor force inequality.

9. Invariant arguments and boundary witnesses — informative

9.1 Preservation and nonreplacement

Fix a lawful P and admissible pre-state. A successful Put(b) either adds a fresh binding at H(b) or writes the equal incumbent. It preserves all old bytes, so every old encoding and span remains valid. R is unchanged.

For successful Admit(r), compatibility preserves old S bindings and supplies S'[c]=E(r), with c=H(E(r)). If c already belongs to R, then:

```
WF             ⇒ S[c] = E(R[c])
compatibility  ⇒ S[c] = E(r)
injective E    ⇒ R[c] = r
```

Thus admission cannot replace a different body. Every new target passed against the pre-state; extension of S leaves its bytes unchanged. Both new and old entries satisfy WF. Read-only successes and all failures preserve the state directly. Induction establishes WF and map extension throughout every finite history from any admissible start. Injectivity of H is unnecessary.

9.2 Every finite admissible state is conditionally constructible

Given finite WF(S*,R*), and conditions enabling all required operations, first Put(S*[a]) for every key a. The addressing law puts each value at exactly its intended key without conflicting with another target binding. Then Admit(R*[c]) for every key c. All sources already exist, and every encoding equals its compatible byte-store value. The result is exactly (S*,R*). This construction requires no topological ordering of record references and makes no unconditional availability claim.

9.3 Empty encoding and cyclic references

Empty encoding. Choose Address=Nat, nonempty bytes x, H(x)=0, H(ε)=1, and r=([(0,0,length(x),0)],ε). The countable body domain permits an injective encoding assigning E(r)=ε and mapping all remaining bodies to distinct nonempty strings. After Put(x), enabled Admit(r) yields a well-formed state with an empty encoding. Nonempty targets do not imply nonempty encoded bytes.

Two-record cycle. Choose Address=Nat, x=61, y=62 in hexadecimal, H(x)=0, H(y)=1, and:

```
r0 = ([(1,0,1,1)], ε)      E(r0) = x
r1 = ([(0,0,1,0)], ε)      E(r1) = y
```

Extend E injectively over the remaining countable body domain and H deterministically over other bytes. Put(x), Put(y), Admit(r0), Admit(r1) produce a cycle under enabling conditions. This is an existence witness for a lawful profile; pre-storing an arbitrary encoding does not solve a self-reference equation for every profile or body.

9.4 State validity is weaker than correct history

An S binding without a corresponding R entry may be well-formed: Put deliberately constructs such states. Exposing only the byte-store effect of a failed admission can therefore preserve WF while violating its atomic transition. Likewise, an empty recovered state satisfies WF but can discard every

committed binding. Recovery that decodes bytes placed by `Put(E(r))` and invents catalogue membership without an admitted or appropriately completed pending `Admit(r)` can preserve both `WF` and map extension while manufacturing history. Conformance needs the transition and history-preservation obligations as well as the state predicate.

10. Operational and security boundaries — informative

Boundary	Consequence
Content identity	Identity alone supplies no authenticity, origin, timestamp, semantic support, or authorization.
Hashing and disclosure	Publicly evaluable digests can permit candidate testing or correlation over small domains. Collisions may leave several candidates; a digest is not a universal unique inversion or confidentiality guarantee.
Occupied addresses	A conflicting retained value can block a future proposed encoding. Access and resource policy affect exposure; the abstract contract does not mandate a cryptographic strength floor.
Untrusted payloads	Sources and statements remain arbitrary bytes. Admission does not sanitize them or authorize a consumer to execute them. Renderer and transport behavior need their own contracts.
Retention and lifecycle	The five operations preserve bindings. Physical compaction may preserve the abstraction; deletion or reinterpretation is not one of these transitions and cannot silently preserve the same retained history.
Profile exchange	Identifier interpretation requires its profile context. The contract defines no negotiation, profile registry, wire envelope, or automatic migration.
Additional interfaces	Search, enumeration, batch operations, point annotations, signatures, diagnostics, and public audit logs are not added by this specification. Any extension needs its own stated semantics and must preserve the core claims it makes.
Claim limits	No performance, universal corruption detection, cryptographic impossibility, confidentiality, semantic accuracy, interoperability, or legal publication effect is established here. The text license supplies no license claim about entrusted source or statement bytes.

A basic realization can retain exact bytes in an associative store, encode bodies injectively, select intervals, and apply the specified transitions. It needs no source-domain parser or inference engine; its correctness depends on its stated retention, comparison, verification, and fault assumptions.